Recovery Subsystem

Mirunalini.P

SSNCE

March 20, 2026

Table of Contents

1. Caching (Buffering) of Disk Blocks
   - Strategies for Writing Modified Buffers to Disk
   - Before Image and After Image
2. Write-Ahead Logging, Steal/No-Steal, and Force/No-Force
   - Types of Log Entry Information
   - Log Buffers in the DBMS Cache

Session Outcome

- Understand Recovery Concepts
- Understand WAL
- Understand Checkpointing
- Understand Deferred Update
- Understand Immediate Update

Recovery Process

- Recovery process restores database to the most **recent consistent state** before the time of failure.
- The system must keep information about the changes that were applied to data items by the various transactions which were kept in system log.
  Typical recovery strategies are:
  - Restore backed-up copy of database from archival storage when the damage due to catastrophic failure
  - Identify any changes that may cause inconsistency in a non-catastrophic failure, reverse the changes by applying either of the operations like **undoing or redo**.

Caching (Buffering) of Disk Blocks

- The recovery process in a DBMS is closely related to operating system functions, especially the buffering of database disk pages in the DBMS main memory cache.
- Multiple disk pages containing data items to be updated are first cached into main memory buffers and then updated in memory before being written back to disk.
- Although caching of disk pages is traditionally an operating system function, in DBMS it is managed by the DBMS itself because it is very important for efficient recovery.
- The DBMS performs this operation by calling low-level operating system routines.
- Recovery is generally considered in terms of database disk pages (blocks).

Caching (Buffering) of Disk Blocks

- A collection of in-memory buffers called the **DBMS cache** is maintained under the control of the DBMS to hold these pages.
- A **cache directory** is maintained to keep track of which database pages are stored in the buffers.
- The cache directory can be represented as a table of entries such as:

  `<Disk_page_address, Buffer_location, ...>`

- When the DBMS needs to access a data item, it first checks the cache directory to see if the disk page containing that item is already in the cache.
- If the page is not in the cache, it must be located on disk and copied into the cache.
- Sometimes existing buffers must be replaced (or flushed) to create space for new pages.

Caching (Buffering) of Disk Blocks

- Each buffer in the cache has a **dirty bit** to indicate whether the page has been modified.
- When a page is first loaded from disk into a buffer, the dirty bit is set to **0**.
- When the page is modified in memory, the dirty bit is set to **1**.
- The cache directory may also store additional information such as the transaction IDs of transactions that modified the buffer.
- When a buffer is replaced or flushed from the cache, its contents must be written back to disk only if the dirty bit is equal to **1**.
- Another control bit called the **pin-unpin bit** is used.
- A page is **pinned** (value 1) when it cannot yet be written back to disk.
- This happens when a transaction is still using the page or when the recovery protocol requires the page to remain in memory until the transaction commits.

Strategies for Writing Modified Buffers to Disk

- Two strategies are used when writing modified buffers back to disk.
- **In-place Updating**
  - The modified buffer is written back to the same original disk location.
  - The old value on disk is overwritten.
  - Only a single copy of each database disk block is maintained.
- **Shadowing**
  - The updated buffer is written to a different disk location.
  - Multiple versions of the data item can be maintained.
  - This method is generally not used in practice.

Before Image and After Image

- The value of a data item before updating is called the **Before Image (BFIM)**.
- The value of a data item after updating is called the **After Image (AFIM)**.
- If shadowing is used, both BFIM and AFIM may exist on disk.
- Therefore, maintaining a separate recovery log may not be strictly necessary.

Write-Ahead Logging, Steal/No-Steal, and Force/No-Force

- When **in-place updating** is used, a log must be maintained to support recovery.
- The recovery mechanism must ensure that the **Before Image (BFIM)** of a data item is recorded in the log before the database on disk is updated.
- The log entry must be written (flushed) to disk before the BFIM in the database is overwritten by the **After Image (AFIM)**.
- This rule is known as **Write-Ahead Logging (WAL)**.
- Write-Ahead Logging ensures that if a failure occurs, the system can **UNDO** the operation during recovery.

- Two types of information are stored in log entries for write operations:
  - Information required for **UNDO**
  - Information required for **REDO**
- A **REDO-type log entry** contains the **new value (AFIM)** of the data item.
- This value is needed to redo the operation during recovery by restoring the item value to its AFIM.
- An **UNDO-type log entry** contains the **old value (BFIM)** of the data item.
- This value is used to undo the effect of an operation by restoring the item value back to its BFIM.
- In an **UNDO/REDO recovery algorithm**, both BFIM and AFIM are stored in the same log entry.

Log Buffers in the DBMS Cache

- The **DBMS cache** stores cached database disk blocks in main memory buffers.
- When a log record is created, it is first written to a **log buffer** in the DBMS cache.
- The log file is maintained as a **sequential (append-only) disk file**. Log File on Disk Record1 $\rightarrow$ Record2 $\rightarrow$ Record3 $\rightarrow$ Record4
- When a data block in the cache is updated, an associated log record is written to the **last log buffer**.
- According to the **Write-Ahead Logging rule**, the log buffers containing the log records must be written to disk before the updated data block is written to disk.

Log Buffers in the DBMS Cache

```
Transaction updates data
        ↓
Data block updated in memory
        ↓
Log record written to LOG BUFFER
        ↓
Log buffer written to LOG FILE (disk)
        ↓
Data block written to DATA FILE (disk)
```

Techniques used to move a page from cache to memory

Standard DBMS recovery terminology includes the terms **steal/no-steal and force/no-force** which specify some rules when a page from database cache can be written to disk:

- **No-Steal Approach:** if a cache page updated by a transaction cannot be written to disk before the transaction commits.
- **Steal Approach:** if the recovery protocol allows writing an update before the transaction commits.
- **Force Approach:** If all pages updated by a transaction are immediately written to disk at the point of transaction commits.
- **No-Force Approach:** Modified pages do not have to be written to disk at commit time. May stay in memory; they are not immediately flushed.
  Typical database systems employ a steal/no-force (UNDO/REDO) strategy and deferred update (NO-UNDO) recovery scheme follows a no-steal approach.

Write Ahead Log [WAL] Rule:

To permit recovery when in-place updating is used, the appropriate entries required for recovery must be permanently recorded in the log on disk before changes are applied to the database
Write-Ahead Logging (WAL) protocol consists of two rules for Undo and Redo operations:

- The BFIM cannot be overwritten by its AFIM in the database on disk until all UNDO-type log records for updating transaction have been force-written to disk.
- The commit operation of a transaction cannot be completed until all the REDO-type and UNDO-type log records for that transaction have been force-written to disk (STABLE STORE).
- The DBMS recovery subsystem may need to maintain a number of lists related to the transactions being processed in the system.
- These include a list for **active transactions** that have started but not committed as yet, and they may also include lists of all committed and aborted transactions since the last checkpoint

CheckPointing

- Checkpoint : Type of entry in the log.
- All their updates will be recorded in the database on disk during checkpointing
- As part of checkpointing, the list of transaction ids for active transactions at the time of the checkpoint is included in the checkpoint record, so that these transactions can be easily identified during recovery.
- The recovery manager of a DBMS must decide at what intervals to take a checkpoint. The interval may be measured in time, every m minutes—or in the number t of committed transactions since the last checkpoint, where the values of m or t are system parameters
- Recovery system used to maintain a list of active transactions,committed and aborted transactions since the last check point.

CheckPointing

- For any system failure, the contents of the main memory was lost.
- When system restarts:
  1. All transactions have their [**commit,T**] enteries in the log before checkpointing will not require any operations, updates done before.
  2. Any transactions that have commited after checkpointing should be redone.
  3. Any transactions that have started before or after checkpoint but not committed should be undone.

CheckPointing

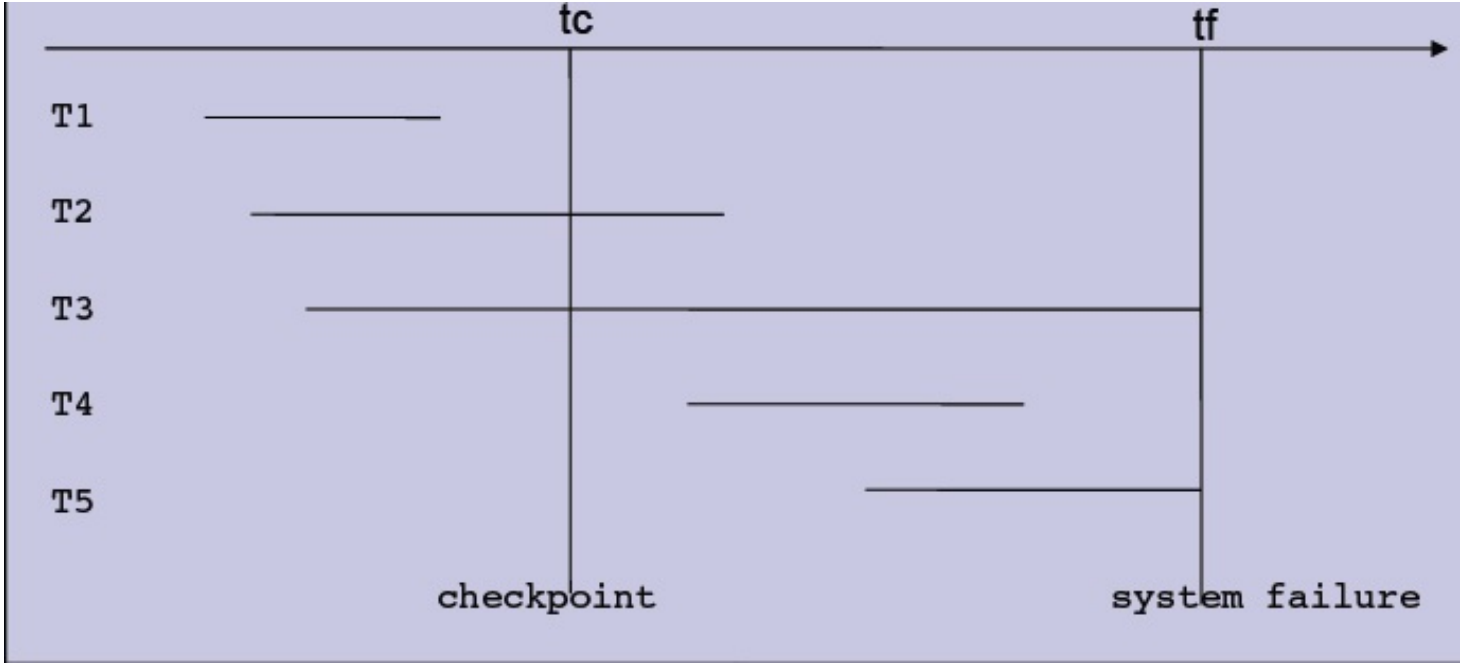

Figure 1: Check pointing - Example

T1 - Committed before Checkpoint, T2,T4 - Committed before system failure, T3, T5 - Not committed still system failure

1. Start with two lists of transactions, the UNDO list an
REDO list.

2. Set UNDO = list of all transactions given in the most

checkpoint record; REDO = empty.

3. Search forward through log, starting from checkpoint r

4. For a transaction T, if a BEGIN TRANSACTION log record

found, add T to the UNDO list.

5. For a transcation T, if a COMMIT log record is found,
from UNDO to REDO list.

6. At the end of the log, the UNDO and REDO lists identi

transactions T3 and T5 has to be UNDONE and

transactions T2 and T4 has to be REDONE.

COMMIT POINT

- A transaction T reaches its commit point when all its operations that access the database have been executed successfully and the effect of all the transaction operations on the database have been recorded in the log.
- Beyond the commit point, the transaction is said to be committed, and its effect must be permanently recorded in the database.
- The transaction then writes a commit record [commit , T] into the log.

Non-Castrotrophic Failure

Two main policies for recovery from non-catastrophic transaction failures: **deferred update and immediate update**.

- **Deferred Update:** Do not physically update the database until after transaction commits.
- Before reaching the commit point, transactions updates are recorded in the log file.
- After commit, updates are written to the database from the main memory buffer.
- If a transaction fails before reaching its commit point, no changes in the database so no **UNDO**.
- After commit REDO the effect of the operations of the committed transaction from the log.
- Deferred update also known as the **NO-UNDO/REDO** algorithm

Deferred Update

Deferred update protocol are as follows:

- A transaction cannot change the database on disk until it reaches its commit point
- A transaction does not reach its commit point until all its REDO -type log entries are recorded in the log and the log buffer is force-written to disk.

Deferred Update Concurrent User

- Concurrency control uses strict two-phase locking, so the locks on items remain in effect until the transaction reaches its commit point.
- After that locks are released.
- Use two lists of transactions maintained by the system: the committed transactions T since the last checkpoint **(commit list)**, and the active transactions T **(active list)**.
- **REDO** all the WRITE operations of the **committed transactions** from the log, in the order in which they were written into the log
- The transactions that are **active and did not commit** are effectively canceled and must be resubmitted.

REDO Procedure

Redoing a write item operation consists of examining its log entry [**write item, T , X , new value**] and setting the value of item X in the database to new value , which is the after image (AFIM)

Deferred Example

(b)

[start_transaction, T_1]
[write_item, T_1, D,20]
[commit, T_1]
[checkpoint]
[start_transaction, T_4]
[write_item, T_4, B,15]
[write_item, T_4, A,20]
[commit, T_4]
[start_transaction, T_2]
[write_item, T_2, B,12]
[start_transaction, T_3]
[write_item, T_3, A,30]
[write_item, T_2, D,25] ← system crash

T_2 and T_3 are ignored because they did not reach their commit points.
T_4 is redone because its commit point is after the last system checkpoint.

Figure 3: Deferred Example

Immediate update

- Immediate Update When a transaction issues an update command, the database can be updated immediately, without any need to wait for transaction to reach its commit point
- Provisions is needed for undoing the effect of update operations that have been applied to the database by a failed transaction.
- Accomplished by rolling back the transaction and undoing the effect of the transactions write item operations.
- UNDO is required because AFIM is flushed to the disk before a transaction commits.
- REDO is required in case the system fails after a transaction commits but before all its changes are recorded in the database on disk.

Immediate update- UNDO/REDO with checkpoints

- Use two lists of transactions maintained by the system: **the committed transactions since the last checkpoint and the active transactions**.
- Undo all the write item operations of the active (uncommitted) transactions, using the UNDO procedure.
- The operations should be undone in the reverse of the order in which they were written into the log.
- Redo all the write item operations of the committed transactions from the log, in the order in which they were written into the log, using the REDO procedure defined earlier.

Procedure UNDO

- Undoing a write item operation consists of examining its log entry [**write item, T , X , old value, new value**] and setting the value of item X in the database to old value , which is the before image (BFIM).
- Undoing from the log must proceed in the reverse order in which the operations were written in the log.